

MODULAR CONTRACTS

Jason Roderick Donaldson Giorgia Piacentino Zichen Zhao

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines: Redundant clauses contradict one another

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines: Redundant clauses contradict one another

Incompleteness

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines: Redundant clauses contradict one another

Incompleteness: Clauses omit relevant contingencies

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines: Redundant clauses contradict one another

Incompleteness: Clauses omit relevant contingencies

Conservatism

FACTS

Current theory: Contract as incentive/sharing rule: outcome $\mapsto$ transfers

Real contracts: Written in language and divided into clauses (“modules”)

Evaluated clause by clause $\Rightarrow$ phenomena not in standard model:

Loopholes: Outcomes can depend on how terms combined

Errors due to complexity: Mistakes parsing long clauses

Landmines: Redundant clauses contradict one another

Incompleteness: Clauses omit relevant contingencies

Conservatism: High burden of proof to verify clause

QUESTIONS

Why are contracts modular, with clauses evaluated separately?

What explains loopholes, complexity, landmines, incompleteness,...?

THIS PAPER

Model of modular contracts

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data:

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

$\equiv$ marginals preserved, not joints

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

$\equiv$ marginals preserved, not joints

E.g. true data: $\mathbf{a}_j = 110, \mathbf{a}_k = 101$

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

$\equiv$ marginals preserved, not joints

E.g. true data: $\mathbf{a}_j = 110, \mathbf{a}_k = 101$; conjunction: $\mathbf{a}_j \wedge \mathbf{a}_k = 100$

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

$\equiv$ marginals preserved, not joints

E.g. true data: $\mathbf{a}_j = 110, \mathbf{a}_k = 101$; conjunction: $\mathbf{a}_j \wedge \mathbf{a}_k = 100$

Observations: $\hat{\mathbf{a}}_j = 110, \hat{\mathbf{a}}_k = 110$

THIS PAPER

Model of modular contracts: Modules = clauses, unit of evaluation

Premises: Atomic terms

Clauses: Groups of premises

Contract: Conjunction of clauses

Evaluation of clauses: 0 or 1 based on threshold of observed data

Assumption: Shuffled data: Observations of premise permutation of data

$\equiv$ marginals preserved, not joints

E.g. true data: $\mathbf{a}_j = 110, \mathbf{a}_k = 101$; conjunction: $\mathbf{a}_j \wedge \mathbf{a}_k = 100$

Observations: $\hat{\mathbf{a}}_j = 110, \hat{\mathbf{a}}_k = 110$; conjunction: $\hat{\mathbf{a}}_j \wedge \hat{\mathbf{a}}_k = 110$

TRADE-OFF: WIDE VS NARROW CLAUSES

TRADE-OFF: WIDE VS NARROW CLAUSES

Wide clauses

TRADE-OFF: WIDE VS NARROW CLAUSES

Wide clauses: Susceptible to shuffling

TRADE-OFF: WIDE VS NARROW CLAUSES

Wide clauses: Susceptible to shuffling

Narrow clauses

TRADE-OFF: WIDE VS NARROW CLAUSES

Wide clauses: Susceptible to shuffling

Narrow clauses: Neglect joint info

RESULTS

RESULTS

Narrow clauses

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

Omitted premises

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

Omitted premises: Coarse proxy of conjunction $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

Omitted premises: Coarse proxy of conjunction $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$

Incompleteness: Cost of coarseness less than risk of shuffling

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

Omitted premises: Coarse proxy of conjunction $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$

Incompleteness: Cost of coarseness less than risk of shuffling

Extension: High thresholds

RESULTS

Narrow clauses: Evaluate $\mathbf{a}_j, \mathbf{a}_k$ one by one to avoid shuffling error

Loopholes: Grouping premises into clauses change outcome

Wide clauses: Keep track of dependencies among premises

Errors due to complexity: Evaluation errors due to shuffled data

Redundant clauses: Repeat same premise $\mathbf{a}_j$ in multiple clauses

Landmines: Internal inconsistencies if premise shuffled in one clause

Omitted premises: Coarse proxy of conjunction $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$

Incompleteness: Cost of coarseness less than risk of shuffling

Extension: High thresholds $\Rightarrow$ conservatism

CONTRIBUTION

Relative to literature on contracting in language (Battigalli–Maggi 02)

Develop framework in which clauses/evaluation matter

Relative to literature on contracting anomalies (e.g. Choi–Gulati–Scott 24)

Develop formal model to rationalize them

MODEL

MODEL

MODEL

(Atomic) Premises

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $\llbracket \mathcal{M}_m \rrbracket = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $[\mathcal{M}_m] = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$; $[\Phi] = 1$ if $\bigwedge_{\mathcal{M}_m \in \Phi} [\mathcal{M}_m] = 1$

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $[\mathcal{M}_m] = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$; $[\Phi] = 1$ if $\bigwedge_{\mathcal{M}_m \in \Phi} [\mathcal{M}_m] = 1$

Target policy

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $\llbracket \mathcal{M}_m \rrbracket = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$; $\llbracket \Phi \rrbracket = 1$ if $\bigwedge_{\mathcal{M}_m \in \Phi} \llbracket \mathcal{M}_m \rrbracket = 1$

Target policy: $\mathcal{T} \subset \mathcal{A}$ evaluated on true data

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $[\mathcal{M}_m] = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$; $[\Phi] = 1$ if $\bigwedge_{\mathcal{M}_m \in \Phi} [\mathcal{M}_m] = 1$

Target policy: $\mathcal{T} \subset \mathcal{A}$ evaluated on true data

Objective

MODEL

(Atomic) Premises: $\mathbf{a}_j \in \mathcal{A}$

Clause: Set of premises $\mathcal{M}_m \subset \mathcal{A}$

Contract: Set of clauses $\Phi = \{\mathcal{M}_m\}_m$

Data: Realizations of $\mathbf{a}_j = (a_j^1 \dots a_j^I) \in \{0, 1\}^I$

Observations = shuffled data: $\hat{\mathbf{a}}_j = \mathbf{a}_j$ w.p. $1 - e$; permutation of $\mathbf{a}_j$ w.p. e

Evaluation: $\llbracket \mathcal{M}_m \rrbracket = 1$ if $\left\| \bigwedge_{\mathbf{a}_j \in \mathcal{M}_m} \hat{\mathbf{a}}_j \right\|_1 \geq \tau$; $\llbracket \Phi \rrbracket = 1$ if $\bigwedge_{\mathcal{M}_m \in \Phi} \llbracket \mathcal{M}_m \rrbracket = 1$

Target policy: $\mathcal{T} \subset \mathcal{A}$ evaluated on true data

Objective: Minimize $c_I \Pr[\llbracket \Phi \rrbracket > \llbracket \mathcal{T} \rrbracket] + c_{II} \Pr[\llbracket \Phi \rrbracket < \llbracket \mathcal{T} \rrbracket]$

MODEL: INDICES AND THEIR SHUFFLING

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

Shuffle risk captures asynchronous reporting

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

Shuffle risk captures asynchronous reporting

States: $a_j^i = 1$ if premise $\mathbf{a}_j$ true in state i

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

Shuffle risk captures asynchronous reporting

States: $a_j^i = 1$ if premise $\mathbf{a}_j$ true in state i

Shuffle risk captures uncertain correlation among premises

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

Shuffle risk captures asynchronous reporting

States: $a_j^i = 1$ if premise $\mathbf{a}_j$ true in state i

Shuffle risk captures uncertain correlation among premises

Criteria: $a_j^i = 1$ if $\mathbf{a}_j$ deemed to satisfy criterion i (e.g. rights or attributes)

MODEL: INDICES AND THEIR SHUFFLING

Agents: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true by agent i (e.g. auditor or judge)

Shuffle risk captures anonymous reports

Dates: $a_j^i = 1$ if $\mathbf{a}_j$ deemed true at date i

Shuffle risk captures asynchronous reporting

States: $a_j^i = 1$ if premise $\mathbf{a}_j$ true in state i

Shuffle risk captures uncertain correlation among premises

Criteria: $a_j^i = 1$ if $\mathbf{a}_j$ deemed to satisfy criterion i (e.g. rights or attributes)

Shuffle risk captures aggregation/data entry errors

DEFINITIONS: PREMISE- AND TARGET-BASED

DEFINITIONS: PREMISE- AND TARGET-BASED

DEFINITIONS: PREMISE- AND TARGET-BASED

Premise-based contract: Narrow clauses: $\Phi^{\mathcal{A}} := \{\{\mathbf{a}_j\}\}_{\mathbf{a}_j \in \mathcal{T}}$

DEFINITIONS: PREMISE- AND TARGET-BASED

Premise-based contract: Narrow clauses: $\Phi^{\mathcal{A}} := \{\{\mathbf{a}_j\}\}_{\mathbf{a}_j \in \mathcal{T}}$

Target-based contract: Wide clauses: $\Phi^{\mathcal{T}} := \{\mathcal{T}\}$

LOOPHOLES

DEFINITION: LOOPHOLE STATES

Loophole states LH: Data realizations s.t. $\llbracket \Phi^{\mathcal{A}} \rrbracket \neq \llbracket \Phi^{\mathcal{T}} \rrbracket$ even if $e = 0$

EXAMPLE: LOOPHOLE

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i
1
2
3

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$
1	1
2	1
3	0

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$
1	1
2	1
3	0
# of 1s $\geq \tau$	

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$
1	1
2	1
3	0
# of 1s $\geq \tau$	1

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	1

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	1

Premise-based: $\llbracket \Phi^{\mathcal{A}} \rrbracket$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	1

Premise-based: $\llbracket \Phi^{\mathcal{A}} \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	1

Premise-based: $[\Phi^A] = [\mathbf{a}_j] \wedge [\mathbf{a}_k] = [110] \wedge [101]$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$
1	1	1
2	1	0
3	0	1
# of 1s $\geq \tau$	1	1

Premise-based: $\llbracket \Phi^A \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket = \llbracket 110 \rrbracket \wedge \llbracket 101 \rrbracket = 1 \wedge 1 = 1$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
# of 1s $\geq \tau$	1	1	

Premise-based: $\llbracket \Phi^A \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket = \llbracket 110 \rrbracket \wedge \llbracket 101 \rrbracket = 1 \wedge 1 = 1$

Target-based:

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
# of 1s $\geq \tau$	1	1	

Premise-based: $\llbracket \Phi^A \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket = \llbracket 110 \rrbracket \wedge \llbracket 101 \rrbracket = 1 \wedge 1 = 1$

Target-based: $\llbracket \Phi^T \rrbracket = \llbracket \mathbf{a}_j \wedge \mathbf{a}_k \rrbracket$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
$\# \text{ of } 1\text{s} \geq \tau$	1	1	

Premise-based: $[\Phi^A] = [\mathbf{a}_j] \wedge [\mathbf{a}_k] = [110] \wedge [101] = 1 \wedge 1 = 1$

Target-based: $[\Phi^T] = [\mathbf{a}_j \wedge \mathbf{a}_k] = [110 \wedge 101]$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
# of 1s $\geq \tau$	1	1	

Premise-based: $[\Phi^A] = [\mathbf{a}_j] \wedge [\mathbf{a}_k] = [110] \wedge [101] = 1 \wedge 1 = 1$

Target-based: $[\Phi^T] = [\mathbf{a}_j \wedge \mathbf{a}_k] = [110 \wedge 101] = [100]$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
# of 1s $\geq \tau$	1	1	0

Premise-based: $\llbracket \Phi^A \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket = \llbracket 110 \rrbracket \wedge \llbracket 101 \rrbracket = 1 \wedge 1 = 1$

Target-based: $\llbracket \Phi^T \rrbracket = \llbracket \mathbf{a}_j \wedge \mathbf{a}_k \rrbracket = \llbracket 110 \wedge 101 \rrbracket = \llbracket 100 \rrbracket = 0$

EXAMPLE: LOOPHOLE

Two premises with three data each and $\tau = 2$:

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
# of 1s $\geq \tau$	1	1	0

Premise-based: $\llbracket \Phi^{\mathcal{A}} \rrbracket = \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket = \llbracket 110 \rrbracket \wedge \llbracket 101 \rrbracket = 1 \wedge 1 = 1$

Target-based: $\llbracket \Phi^{\mathcal{T}} \rrbracket = \llbracket \mathbf{a}_j \wedge \mathbf{a}_k \rrbracket = \llbracket 110 \wedge 101 \rrbracket = \llbracket 100 \rrbracket = 0$

Loophole: Evaluation & conjunction don't commute: $\llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket \neq \llbracket \mathbf{a}_j \wedge \mathbf{a}_k \rrbracket$

LOOPHOLES IN PRACTICE: J. CREW

J. Crew was distressed and wanted to free up collateral

Credit agreement ostensibly banned operations exploiting creditors, but...

Clause $\mathbf{a}_j$: “Can move assets to restricted sub”

Clause $\mathbf{a}_k$: “Can move assets from restricted to unrestricted sub”

LOOPHOLES IN PRACTICE: J. CREW

J. Crew was distressed and wanted to free up collateral

Credit agreement ostensibly banned operations exploiting creditors, but...

Clause $\mathbf{a}_j$: “Can move assets to restricted sub”

Clause $\mathbf{a}_k$: “Can move assets from restricted to unrestricted sub”

$\Rightarrow [\mathbf{a}_j] \wedge [\mathbf{a}_k]$: “Can move assets to unrestricted sub [via restricted]”

LOOPHOLES IN PRACTICE: J. CREW

J. Crew was distressed and wanted to free up collateral

Credit agreement ostensibly banned operations exploiting creditors, but...

Clause $\mathbf{a}_j$: “Can move assets to restricted sub”

Clause $\mathbf{a}_k$: “Can move assets from restricted to unrestricted sub”

$\Rightarrow \llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket$: “Can move assets to unrestricted sub [via restricted]”

I.e. loophole: $\llbracket \mathbf{a}_j \rrbracket \wedge \llbracket \mathbf{a}_k \rrbracket \neq \llbracket \mathbf{a}_j \wedge \mathbf{a}_k \rrbracket$

SHUFFLE RISK

DEFINITION: SHUFFLE RISK STATES

Shuffle risk states SR: Data realizations s.t. $0 < \Pr\left[\llbracket \Phi^T \rrbracket = 1 \mid \{\mathbf{a}_j\}_j\right] < 1$

EXAMPLE: SHUFFLE RISK

Observe permuted data

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$
1	1	1	1
2	1	0	0
3	0	1	0
$\# \text{ of } 1\text{s} \geq \tau$	1	1	0

EXAMPLE: SHUFFLE RISK

Observe permuted data E.g. $\hat{\mathbf{a}}_j = \mathbf{a}_j$ (no shuffle)

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$	$\hat{\mathbf{a}}_j$
1	1	1	1	1
2	1	0	0	1
3	0	1	0	0
# of 1s $\geq \tau$	1	1	0	1

EXAMPLE: SHUFFLE RISK

Observe permuted data E.g. $\hat{\mathbf{a}}_j = \mathbf{a}_j$ (no shuffle) ; $\hat{\mathbf{a}}_k = 110$ (shuffle of $\mathbf{a}_k$)

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$	$\hat{\mathbf{a}}_j$	$\hat{\mathbf{a}}_k$
1	1	1	1	1	1
2	1	0	0	1	1
3	0	1	0	0	0
# of 1s $\geq \tau$	1	1	0	1	1

EXAMPLE: SHUFFLE RISK

Observe permuted data E.g. $\hat{\mathbf{a}}_j = \mathbf{a}_j$ (no shuffle) ; $\hat{\mathbf{a}}_k = 110$ (shuffle of $\mathbf{a}_k$)

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$	$\hat{\mathbf{a}}_j$	$\hat{\mathbf{a}}_k$
1	1	1	1	1	1
2	1	0	0	1	1
3	0	1	0	0	0
# of 1s $\geq \tau$	1	1	0	1	1

Marginals preserved $\Rightarrow$ premise-based evaluation unchanged

EXAMPLE: SHUFFLE RISK

Observe permuted data E.g. $\hat{\mathbf{a}}_j = \mathbf{a}_j$ (no shuffle) ; $\hat{\mathbf{a}}_k = 110$ (shuffle of $\mathbf{a}_k$)

i	$\mathbf{a}_j$	$\mathbf{a}_k$	$\mathbf{a}_j \wedge \mathbf{a}_k$	$\hat{\mathbf{a}}_j$	$\hat{\mathbf{a}}_k$	$\hat{\mathbf{a}}_j \wedge \hat{\mathbf{a}}_k$
1	1	1	1	1	1	1
2	1	0	0	1	1	1
3	0	1	0	0	0	0
# of 1s $\geq \tau$	1	1	0	1	1	1

Marginals preserved $\Rightarrow$ premise-based evaluation unchanged

Joint changed: $\mathbf{a}_k \rightarrow \hat{\mathbf{a}}_k \Rightarrow$ target-based evaluation flips

RESULTS

R1: LOOPHOLES

R1: LOOPHOLES

With $e > 0$, if $\Pr[\text{SR} \setminus \text{LH}]$ large, then $\Phi^{\mathcal{A}} \succ \Phi^{\mathcal{T}}$

R1: LOOPHOLES: INTUITION

If loopholes are infrequent, make clauses narrow

R1: LOOPHOLES: INTUITION

If loopholes are infrequent, make clauses narrow

Accept loopholes to reduce shuffling error

LOOPHOLES IN THE LITERATURE

Katz 10: Many loopholes openly exploited after much litigation

LOOPHOLES IN THE LITERATURE

Katz 10: Many loopholes openly exploited after much litigation

Need theory of inherent logical structure, not mistakes

LOOPHOLES IN THE LITERATURE

Katz 10: Many loopholes openly exploited after much litigation

Need theory of inherent logical structure, not mistakes

Suggests connection with Arrow's thm

Doctrinal Paradox links Arrow to logic/formal language (e.g. List–Pettit 02)

R2: ERRORS DUE TO COMPLEXITY

R2: ERRORS DUE TO COMPLEXITY

If $c_I \Pr[\text{LH}]$ sufficiently high, $\Phi^{\mathcal{T}} \succ \Phi^{\mathcal{A}}$

R2: COMPLEXITY RISK: INTUITION

If loopholes likely or costly, use wide clauses

R2: COMPLEXITY RISK: INTUITION

If loopholes likely or costly, use wide clauses

Cost: Complexity. With interacting premises, likely evaluated with error

R2: COMPLEXITY RISK: INTUITION

If loopholes likely or costly, use wide clauses

Cost: Complexity. With interacting premises, likely evaluated with error

Empirically: Complexity leads to errors in taxes and regulatory compliance
OECD, Benzarti 20, Zwick 21, Behn et al 22...

R3: REDUNDANCY

DEFINITION: REDUNDANCY

$\Phi^{\mathcal{R}}$ called redundant if one premise in multiple clauses

I.e. if there's $\mathbf{a}_j$ s.t. $\mathbf{a}_j \in \mathcal{M}_m \cap \mathcal{M}_{m'}$ for clauses $m \neq m'$

R3: REDUNDANCY

If $e > 0$ not too high and $c_1\text{Pr}[\text{LH}]$ high enough, both $\Phi^{\mathcal{R}} \succ \Phi^{\mathcal{T}}$ & $\Phi^{\mathcal{R}} \succ \Phi^{\mathcal{A}}$

R3: REDUNDANCY: INTUITION

Benefit w.r.t. premise-based: Keep joint information

R3: REDUNDANCY: INTUITION

Benefit w.r.t. premise-based: Keep joint information

Benefit w.r.t. target-based: Protect against shuffling errors

R3: REDUNDANCY: INTUITION

Benefit w.r.t. premise-based: Keep joint information

Benefit w.r.t. target-based: Protect against shuffling errors

Cost: False negatives

R4: INCOMPLETENESS

DEFINITION: INCOMPLETENESS

$\Phi^{\mathcal{I}}$ called incomplete w.r.t. target if omits premise from target

I.e. if $\mathbf{a}_j \notin \bigcup_{\mathcal{M}_m \in \Phi^{\mathcal{I}}} \mathcal{M}_m$ for some $\mathbf{a}_j \in \mathcal{T}$

R4: INCOMPLETENESS

“Sometimes” $\Phi^{\mathcal{I}} \succ \Phi^{\mathcal{T}} \ \& \ \Phi^{\mathcal{I}} \succ \Phi^{\mathcal{A}}$

R4: INCOMPLETENESS: INTUITION

Benefit w.r.t. target-based: Protect against shuffling errors

R4: INCOMPLETENESS: INTUITION

Benefit w.r.t. target-based: Protect against shuffling errors

Benefit w.r.t. premise-based: If “proxy task” $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$: Keep joint info

R4: INCOMPLETENESS: INTUITION

Benefit w.r.t. target-based: Protect against shuffling errors

Benefit w.r.t. premise-based: If “proxy task” $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$: Keep joint info

Akin to mismeasurement in Holmström–Milgrom:

R4: INCOMPLETENESS: INTUITION

Benefit w.r.t. target-based: Protect against shuffling errors

Benefit w.r.t. premise-based: If “proxy task” $\mathbf{a}_\ell \approx \mathbf{a}_j \wedge \mathbf{a}_k$: Keep joint info

Akin to mismeasurement in Holmström–Milgrom:

Decrease power on distortionary tasks via omission or proxy

R5: CONSERVATISM

EXTENSION

Clause-dependent thresholds $\tau_{\mathcal{M}}$

DEFINITION: CONSERVATISM

Let $\tau_{\mathcal{T}}$ be threshold of target

$\Phi^{\mathcal{C}}$ is conservative if $\tau_{\mathcal{M}_m} > \tau_{\mathcal{T}}$ for every m

R5: CONSERVATISM (INFORMALLY)

“Often” $\Phi^{\mathcal{C}} \succ \Phi^{\mathcal{A}}$ and $\Phi^{\mathcal{C}} \succ \Phi^{\mathcal{T}}$

R5: CONSERVATISM: INTUITION

Premise-based gives false positives; conservatism decreases them

R5: CONSERVATISM: INTUITION

Premise-based gives false positives; conservatism decreases them

Feature: Relies on only marginals $\Rightarrow$ not subject to shuffle risk

R5: CONSERVATISM: INTUITION

Premise-based gives false positives; conservatism decreases them

Feature: Relies on only marginals $\Rightarrow$ not subject to shuffle risk

Bug (lite): Still neglects joint info (but fewer false positives than $\Phi^{\mathcal{A}}$)

R5: CONSERVATISM: APPLICATIONS

Law: “beyond reasonable doubt,” “reasonable suspicion,” “probable cause”
In re Winship, 397 U.S. 358 (1970), Terry v. Ohio, 392 U.S. 1 (1968)

Accounting & auditing: High standard for good news Basu 97; Watts 03

E.g. losses recognized faster than gains

CONCLUSION

CONCLUSION

Model of modular contracts in formal language s.t. shuffled data

Loopholes

Errors of complexity

Landmines

Incompleteness

Conservatism

MODULAR CONTRACTS